FinShare — Abhilash: What To Do & Where

Every pending task organized by file — with NEW FILE vs MODIFY clearly marked

April 3, 2026 | .NET 8 | SQL Server | Dapper

NEW FILE	MODIFY	SQL — NEW TABLE	SQL — NEW SP	CONFIG
Create a new .cs file	*Add code to existing file*	*Run in SQL Server*	*Run in SQL Server*	*Server / file config*

📌 **HOW TO READ THIS — Each block shows the FILE NAME with a tag telling you whether to CREATE a new file or MODIFY an existing one. Under each file is the list of things to add/change in that file. Complete Phase 1 fully before starting Phase 2.**

PHASE 1 — Authentication & Security (Start Here)

📄 **Full code for everything below is in: FinShare_ForAbhilash_AuthSecurity_v3.docx and FinShare_ForAbhilash_MASTER.docx**

SQL — NEW TABLE ALTER TABLE Users *(add 9 new columns)*

- EmailVerificationToken NVARCHAR(200) NULL
- EmailVerifyExpiry DATETIME NULL
- IsEmailVerified BIT NOT NULL DEFAULT 0
- PasswordResetToken NVARCHAR(200) NULL
- PasswordResetExpiry DATETIME NULL
- LoginAttempts INT NOT NULL DEFAULT 0
- LockoutUntil DATETIME NULL
- WebAuthnCredentialId NVARCHAR(500) NULL
- WebAuthnPublicKey NVARCHAR(MAX) NULL
- WebAuthnSignCount INT NULL
- WebAuthnAaGuid NVARCHAR(100) NULL

SQL — NEW SP sp_SaveEmailVerifyToken

- Saves the email verification token and expiry for a member (UPDATE Users SET ... WHERE MemberId = @MemberId)

SQL — NEW SP sp_VerifyEmail

- Checks token is valid and not expired, then sets IsEmailVerified = 1 and clears the token

SQL — NEW SP sp_SavePasswordResetToken

- Saves the password reset token and expiry for a member

SQL — NEW SP sp_ResetPassword

- Checks token is valid and not expired, updates the password hash, clears the reset token

SQL — NEW SP sp_RecordFailedLogin

- Increments LoginAttempts for a member. If LoginAttempts >= 5, sets LockoutUntil = GETUTCDATE() + 15 minutes

SQL — NEW SP sp_ResetLoginAttempts

- Sets LoginAttempts = 0 and LockoutUntil = NULL after a successful login

NEW FILE PasswordValidator.cs *(Services folder)*

- Static method: Validate(string password) → returns list of errors
- Rules: minimum 8 characters, at least 1 uppercase, 1 lowercase, 1 digit, 1 special character (!@#$%^&*)
- Returns empty list if password is valid, list of error strings if not

NEW FILE EmailService.cs *(Services folder)*

- Method: SendVerificationEmail(string toEmail, string token)
- Method: SendPasswordResetEmail(string toEmail, string token)
- Use SendGrid (or SMTP) — configure API key in appsettings.Production.json
- Verification link format: https://fintrack.sysdatacenter.com/verify-email?token=TOKEN
- Reset link format: https://fintrack.sysdatacenter.com/reset-password?token=TOKEN

MODIFY AuthController.cs *(existing file)*

- Register endpoint — after saving user, call sp_SaveEmailVerifyToken + EmailService.SendVerificationEmail
- Register endpoint — apply PasswordValidator.Validate() on the password before saving. Return errors if invalid
- Login endpoint — check IsEmailVerified = 1 before issuing JWT. Return 401 with message 'Please verify your email first' if not verified
- Login endpoint — check LockoutUntil. If lockout is active, return 401 with 'Account locked. Try again after X minutes'
- Login endpoint — on failed password: call sp_RecordFailedLogin
- Login endpoint — on success: call sp_ResetLoginAttempts
- ADD new endpoint: POST /api/Auth/verify-email — reads token from query/body, calls sp_VerifyEmail
- ADD new endpoint: POST /api/Auth/resend-verification — generates new token, calls sp_SaveEmailVerifyToken + sends email
- ADD new endpoint: POST /api/Auth/forgot-password — finds member by email, generates reset token, calls sp_SavePasswordResetToken + EmailService.SendPasswordResetEmail
- ADD new endpoint: POST /api/Auth/reset-password — calls sp_ResetPassword, applies PasswordValidator on new password

MODIFY Program.cs *(existing file)*

- Register EmailService as a singleton or scoped service: builder.Services.AddScoped<EmailService>()
- Register PasswordValidator (if class-based, otherwise static — no registration needed)

CONFIG appsettings.Production.json *(on the server only — not in Git)*

- Add email service credentials: SendGrid API key OR SMTP host/port/username/password
- Add JWT secret (already discussed in separate doc)

📌 **WebAuthn (Biometric Login) — Only start this after all other Phase 1 items above are working and tested.**

SQL — NEW SP sp_SaveWebAuthnCredential

- Saves CredentialId, PublicKey, SignCount, AaGuid for a member (UPDATE Users SET ...)

SQL — NEW SP sp_GetWebAuthnCredential

- Returns the WebAuthn credential for a member by MemberId or CredentialId

SQL — NEW SP sp_UpdateWebAuthnSignCount

- Updates the SignCount after each successful biometric login (replay attack prevention)

MODIFY AuthController.cs *(add 4 new WebAuthn endpoints)*

- ADD: POST /api/Auth/webauthn/register-options — returns challenge for the browser to use with navigator.credentials.create()
- ADD: POST /api/Auth/webauthn/register-verify — verifies the credential from the browser, calls sp_SaveWebAuthnCredential
- ADD: POST /api/Auth/webauthn/login-options — returns challenge for the browser to use with navigator.credentials.get()
- ADD: POST /api/Auth/webauthn/login-verify — verifies assertion, checks SignCount, calls sp_UpdateWebAuthnSignCount, issues JWT

MODIFY Program.cs *(add Fido2NetLib)*

- Install NuGet: Fido2NetLib
- Register: builder.Services.AddFido2(options => { options.ServerDomain = 'fintrack.sysdatacenter.com'; options.ServerName = 'FinShare'; })

PHASE 2 — Security Hardening (After Phase 1 is Working)

Full code for everything below is in: FinShare_ForAbhilash_SecurityPhase2.docx

SQL — NEW TABLE CREATE TABLE AuditLogs

- Columns: AuditId (PK), EventType NVARCHAR(50), MemberId INT NULL, IPAddress NVARCHAR(50), Details NVARCHAR(500), CreatedAt DATETIME DEFAULT GETUTCDATE()

SQL — NEW TABLE CREATE TABLE UserRefreshTokens

- Columns: TokenId (PK), MemberId INT FK, Token NVARCHAR(500), ExpiresAt DATETIME, IsRevoked BIT DEFAULT 0, CreatedAt DATETIME DEFAULT GETUTCDATE()

SQL — NEW SP sp_WriteAuditLog

- INSERT INTO AuditLogs — called from C# whenever a security event happens

SQL — NEW SP sp_CleanupExpiredTokens

- DELETE expired email verify tokens, password reset tokens, and refresh tokens older than their expiry

SQL — NEW SP sp_SaveRefreshToken

- INSERT a new refresh token for a member

SQL — NEW SP sp_ValidateRefreshToken

- SELECT token where Token = @Token AND IsRevoked = 0 AND ExpiresAt > GETUTCDATE()

SQL — NEW SP sp_RevokeRefreshToken

- UPDATE UserRefreshTokens SET IsRevoked = 1 WHERE Token = @Token

SQL — NEW SP sp_RevokeAllUserRefreshTokens

- UPDATE UserRefreshTokens SET IsRevoked = 1 WHERE MemberId = @MemberId — used when password changes

MODIFY Program.cs *(add rate limiting + forwarded headers)*

- Add app.UseForwardedHeaders() near the top — needed to get the real client IP behind a proxy/load balancer
- Add builder.Services.AddRateLimiter() — login: 5 attempts per IP per 15 min. Register: 3 per IP per hour
- Add app.UseRateLimiter() in the middleware pipeline

NEW FILE IpHelper.cs *(Helpers folder)*

- Static method: GetClientIP(HttpContext context) — reads X-Forwarded-For header first, falls back to RemoteIpAddress

MODIFY AuthController.cs *(add audit logs + refresh token endpoints)*

- In Login: call sp_WriteAuditLog for LOGIN_SUCCESS and LOGIN_FAILED events, passing IP from IpHelper.GetClientIP()
- In Login success: call sp_SaveRefreshToken, set token as httpOnly cookie in the response
- ADD new endpoint: POST /api/Auth/refresh — read refresh token from httpOnly cookie, call sp_ValidateRefreshToken, issue new access token + new refresh token, revoke old one
- ADD new endpoint: POST /api/Auth/revoke — call sp_RevokeRefreshToken on logout

CONFIG SQL Agent Job or .NET BackgroundService

- Set up a daily job to run sp_CleanupExpiredTokens — can be SQL Agent job in SQL Server or a BackgroundService class in .NET

CONFIG appsettings.Production.json *(on the server — not in Git)*

- Confirm JWT secret is here and NOT in appsettings.json (see FinShare_ForAbhilash_JWTSecretStorage.docx)
- Confirm Encryption:Key and Encryption:IV are here (for Phase 4 Bank Details)

PHASE 3 — New Features: Profile, Transactions, Notifications

Full code for everything below is in: FinShare_ForAbhilash_NewFeatures.docx

SQL — NEW TABLE ALTER TABLE Users *(add 1 new column)*

- ProfilePhotoUrl NVARCHAR(500) NULL

SQL — NEW TABLE CREATE TABLE Notifications

- Columns: NotificationId BIGINT IDENTITY PK, MemberId INT FK, Title NVARCHAR(100), Body NVARCHAR(300), Type NVARCHAR(30), ReferenceId INT NULL, ReferenceType NVARCHAR(20) NULL, IsRead BIT DEFAULT 0, CreatedAt DATETIME DEFAULT GETUTCDATE()

SQL — NEW SP sp_GetUserProfile

- SELECT MemberId, MemberName, UserName, EmailAddress, Mobile, ProfilePhotoUrl, IsEmailVerified FROM Users WHERE MemberId = @MemberId

SQL — NEW SP sp_UpdateUserProfile

- UPDATE Users SET MemberName = @MemberName, Mobile = @Mobile WHERE MemberId = @MemberId

SQL — NEW SP sp_UpdateProfilePhoto

- UPDATE Users SET ProfilePhotoUrl = @ProfilePhotoUrl WHERE MemberId = @MemberId

SQL — NEW SP sp_ChangePassword

- SELECT PasswordHash from Users for verification, then UPDATE PasswordHash WHERE MemberId = @MemberId

SQL — NEW SP sp_GetTransactionHistory

- UNION ALL of: group expenses (JOIN ExpenseMembers), personal expenses, settlements — with filter params @DateFrom, @DateTo, @Type, @GroupId, @Category and pagination @Page, @PageSize

SQL — NEW SP sp_GetTransactionSummary

- Returns 3 numbers: TotalSpent (group + personal), TotalToReceive (settlements where PaidBy = @MemberId and status Pending), TotalToPay (settlements where PaidTo = @MemberId and status Pending)

SQL — NEW SP sp_CreateNotification

- INSERT INTO Notifications

SQL — NEW SP sp_GetNotifications

- Returns UnreadCount (COUNT WHERE IsRead=0) + paginated notifications for @MemberId, ordered by CreatedAt DESC

SQL — NEW SP sp_MarkNotificationRead

- UPDATE Notifications SET IsRead = 1 WHERE NotificationId = @NotificationId AND MemberId = @MemberId

SQL — NEW SP sp_MarkAllNotificationsRead

- UPDATE Notifications SET IsRead = 1 WHERE MemberId = @MemberId

NEW FILE UserController.cs

- GET /api/User/profile — calls sp_GetUserProfile
- PUT /api/User/profile — calls sp_UpdateUserProfile
- POST /api/User/profile/photo — validates image (type + max 2MB), resizes to 200×200 using SixLabors.ImageSharp, saves to /uploads/avatars/{memberId}.jpg, calls sp_UpdateProfilePhoto
- PUT /api/User/change-password — BCrypt verify current password, PasswordValidator on new, calls sp_ChangePassword

NEW FILE TransactionController.cs

- GET /api/Transaction/history — reads query params (from, to, type, groupId, category, page, pageSize), calls sp_GetTransactionHistory + sp_GetTransactionSummary, returns summary + paginated list

NEW FILE NotificationController.cs

- GET /api/Notifications — calls sp_GetNotifications
- PUT /api/Notifications/{id}/read — calls sp_MarkNotificationRead
- PUT /api/Notifications/read-all — calls sp_MarkAllNotificationsRead

MODIFY ExpensesController.cs (or wherever AddExpense / EditExpense is)

- After saving a new expense: call sp_CreateNotification for each group member (Type = EXPENSE_ADDED)
- After editing an expense: call sp_CreateNotification for each group member (Type = EXPENSE_EDITED)

MODIFY SettlementController.cs

- After creating a settlement request: call sp_CreateNotification for the person who needs to pay (Type = SETTLEMENT_REQUEST)

MODIFY Program.cs

- Install NuGet: SixLabors.ImageSharp
- Register NotificationController, UserController, TransactionController (auto-registered if using AddControllers())

PHASE 4 — Bank Details

Full code for everything below is in: FinShare_ForAbhilash_BankDetails.docx

SQL — NEW TABLE CREATE TABLE MemberBankDetails

- Columns: BankDetailId INT PK, MemberId INT FK UNIQUE, AccountHolderName NVARCHAR(150), IBANEncrypted NVARCHAR(500), BankName NVARCHAR(150), Branch NVARCHAR(150) NULL, CreatedAt DATETIME, UpdatedAt DATETIME

SQL — NEW SP sp_SaveBankDetails *(MERGE upsert — insert or update in one call)*

SQL — NEW SP sp_GetBankDetails

SQL — NEW SP sp_GetMemberBankDetailsForSettlement *(only if caller is the payer of that settlement)*

SQL — NEW SP sp_DeleteBankDetails

NEW FILE EncryptionService.cs *(Services folder)*

- Encrypt(string plainText) → returns base64 string using AES-256
- Decrypt(string cipherText) → returns original plain text
- MaskIBAN(string plainIBAN) → returns '•••• •••• •••• XXXX' (last 4 chars only)
- Reads Key (32 chars) and IV (16 chars) from configuration — store in appsettings.Production.json ONLY

MODIFY UserController.cs *(add bank detail endpoints)*

- GET /api/User/bank-details — decrypt IBAN in C#, return full IBAN + masked IBAN
- PUT /api/User/bank-details — encrypt IBAN in C# before calling sp_SaveBankDetails
- DELETE /api/User/bank-details — calls sp_DeleteBankDetails

✏️ **MODIFY** **SettlementController.cs** *(add payee bank details endpoint)*

- GET /api/Settlement/{settlementId}/payee-bank-details — calls sp_GetMemberBankDetailsForSettlement, decrypt IBAN, return 403 if caller is not the payer

✏️ **MODIFY** **Program.cs**

- Register EncryptionService as Singleton: builder.Services.AddSingleton<EncryptionService>()

⚙️ **CONFIG** **appsettings.Production.json** *(on the server — not in Git)*

- Add: Encryption:Key (exactly 32 characters — generate a random one)
- Add: Encryption:IV (exactly 16 characters — generate a random one)

PHASE 5 — Group Events (Only after Joseph confirms Phase 2 is done)

📄 **Full code for everything below is in: FinShare_ForAbhilash_EventBackend.docx | Frontend already works with localStorage — this phase makes events visible to ALL group members.**

🗄️ **SQL — NEW TABLE** **CREATE TABLE GroupEvents**

- Columns: EventId INT PK, GroupId INT FK, CreatedBy INT FK, Title NVARCHAR(150), EventDate DATE, EventTime NVARCHAR(5) NULL, Location NVARCHAR(200) NULL, Notes NVARCHAR(500) NULL, CreatedAt DATETIME, UpdatedAt DATETIME

🗄️ **SQL — NEW SP** **sp_GetGroupEvents** *(verify caller is a group member before returning)*

🗄️ **SQL — NEW SP** **sp_CreateGroupEvent** *(verify caller is a group member)*

🗄️ **SQL — NEW SP** **sp_UpdateGroupEvent** *(only the creator can update — check CreatedBy = @MemberId)*

🗄️ **SQL — NEW SP** **sp_DeleteGroupEvent** *(only the creator can delete — check CreatedBy = @MemberId)*

🆕 **NEW FILE** **GroupEventsController.cs**

- GET /api/Group/{groupId}/events — calls sp_GetGroupEvents
- POST /api/Group/{groupId}/events — calls sp_CreateGroupEvent, returns new EventId
- PUT /api/Group/{groupId}/events/{eventId} — calls sp_UpdateGroupEvent, 403 if not creator
- DELETE /api/Group/{groupId}/events/{eventId} — calls sp_DeleteGroupEvent, 403 if not creator

— Joseph Xavier | FinShare Project | April 3, 2026 | Detailed code for each item is in the phase-specific docs already sent